

Memoria SpotESlfy

FASE 1 - DIAGNOSTICO

E1 — Web no accesible a través del proxy

Sintoma:

Al revisar los logs de nginx, nginx mostraba el error: host not found in upstream "backend:5000"

Ademas el checker mostraba que la web no era accesible a traves del proxy

Diagnostico:

La causa era que en la configuracion de nginx se intentaba enviar el trafico de la api al host backend:5000, pero no existe ningun servicio llamado asi, el servicio real es api.

Solucion:

Se ha modificado la configuracion de nginx, sustituyendo backend:5000 por api:5000

Concepto de SS00:

El error esta relacionado con redes en contenedores y la configuracion de un proxy inverso

E2 — API no conecta con la base de datos

Sintoma:

Al revisar los logs de api este aparecia "killed", luego "getaddrinfo ENOTFOUND db" y finalmente "NOAUTH Authentication required"

Diagnostico:

El problema tenia varias causas, el primero era que api tenia un limite de memoria demasiado bajo y el proceso era terminado

Despues la api y redis estaban en redes distintas por lo que no se podia resolver el nombre db

Finalmente redis exigia contraseña pero la api usaba la variable "REDIS_PASS" mientras que en el archivo docker-compose.yml estaba definida como "REDIS_PASSWORD"

Solucion:

Primero se aumento el limite de memoria de api luego se conecto a la red backend y para terminar se cambio la variable de "REDIT_PASSWORD" a "REDIS_PASS"

```
alumno@spotesify-vm .../ spotesify on main
> docker inspect spotesify-api --format '{{.State.ExitCode}} {{.State.OOMKilled}} {{.HostConfig.Memory}}'
0 false 268435456
```

Concepto de SS00:

Gestion de memoria, limite de recursos en contenedores, redes virtuales Docker, variables de entorno y comunicacion cliente-servidor

E3 — Catálogo de canciones accesible

Sintoma:

La ruta de la api encargada de listar las canciones no devolvía correctamente las 5 canciones

Diagnostico:

Por culpa del error "E2" api no podía conectarse ni autenticarse contra Redis y por tanto no podía leer las canciones. Redis es el almacen donde se guardan las canciones.

Solucion:

Al corregir el problema de api, este se pudo conectar a Redis y leer las canciones que este contenía.

```
alumno@spotefisy-vm ~/$ spotefisy on main
> curl http://localhost/api/songs
[{"id":"genxbeats-dungeon-type-rap-beat-20241116-300651","title":"Genxbeats Dungeon Type Rap Beat 20241116 300651","artist":"Desconocido","album":"Spotefisy","duration":0,"genre":"","filename":"genxbeats-dungeon-type-rap-beat-20241116-300651.mp3"},{"id":"kmaled-attic-secrets-162040","title":"Kmaled Attic Secrets 162040","artist":"Desconocido","album":"Spotefisy","duration":0,"genre":"","filename":"kmaled-attic-secrets-162040.mp3"},{"id":"mortaz-colorful-autumn-454654","title":"Mortaz Colorful Autumn 454654","artist":"Desconocido","album":"Spotefisy","duration":0,"genre":"","filename":"mortaz-colorful-autumn-454654.mp3"},{"id":"mortaz-fading-colors-456885","title":"Mortaz Fading Colors 456885","artist":"Desconocido","album":"Spotefisy","duration":0,"genre":"","filename":"mortaz-fading-colors-456885.mp3"},{"id":"sonikai-cyberpunk-gaming-rave-no-copyright-music-robopanda-285862","title":"Sonikai Cyberpunk Gaming Rave No Copyright Music Robopanda 285862","artist":"Desconocido","album":"Spotefisy","duration":0,"genre":"","filename":"sonikai-cyberpunk-gaming-rave-no-copyright-music-robopanda-285862.mp3"}]
```

Concepto de SS00:

Comunicación entre procesos distribuidos, persistencia de datos en un servicio externo y arquitectura cliente-servidor

E4 — Streaming de audio falla

Sintoma:

El catálogo de canciones era accesible, pero al probar el streaming de audio el servidor respondía con HTTP 404. Al comprobar /data/music dentro del contenedor, el nombre de los ficheros no coincidían.

Diagnostico:

El problema era la inconsistencia de los datos almacenados en Redis y los ficheros reales, ya que en el directorio /data/music existían los ficheros pero con otros nombres. Por tanto api encontraba la canción pero no los ficheros físicos para reproducirlos

Solucion:

Se vació Redis y se reinició la api para que ejecutara de nuevo el proceso de auto-seed. Este proceso apunta al directorio /data/music y rellena la nueva lista con los ficheros que si existen. Además tuvimos que cambiar el volumen en el que estaba montado ya que este era ./music:./music y el correcto es ./music:/data/music

```
alumno@spotefisy-vm ~/$ spotefisy on main
> curl -I http://localhost/api/stream/genxbeats-dungeon-type-rap-beat-20241116-300651
HTTP/1.1 200 OK
Server: nginx/1.29.8
Date: Sat, 09 May 2026 12:30:40 GMT
Content-Type: audio/mpeg
Content-Length: 3596120
Connection: keep-alive
X-Powered-By: Express
Accept-Ranges: bytes
```

Concepto de SS00:

Sistemas de ficheros, montaje de volúmenes y acceso de ficheros desde contenedores

E5 — Contenedor API estable

Sintoma:

Al revisar los logs de api este aparecía constantemente con el mensaje "killed", lo que impedía que el servicio fuera estable

Diagnostico:

La causa era un límite de memoria demasiado bajo configurado para el contenedor api, ya que su límite eran nada más 15MB

Solución:

Se modificó el límite de memoria aumentándolo a 256MB. Después se volvió a construir la imagen y reiniciar los servicios

```
alumno@spotesify-vm ~/spotesify on main
> docker inspect spotesify-api --format 'RestartCount={{.RestartCount}} Memory={{.HostConfig.Memory}}'
RestartCount=0 Memory=268435456
```

Concepto de SS00:

Gestión de memoria y límite de recursos en contenedores

E6 — Imagen no optimizada

Sintoma:

El checker indicaba que la imagen no estaba optimizada pesando 383MB y que el proceso se ejecutaba como root

Diagnostico:

La causa era el Dockerfile de api, la imagen usaba una imagen demasiado pesada y no especificaba USER, por lo que el proceso se ejecutaba con privilegios de root dentro del contenedor.

Solución:

Se modificó el Dockerfile de la api para usar node:20-alpine como imagen la cual pesa bastante menos y ejecutar el proceso como el usuario node mediante USER node

```
alumno@spotesify-vm ~/spotesify on main
> docker compose images
docker compose exec api whoami
CONTAINER    REPOSITORY    TAG        PLATFORM    IMAGE ID        SIZE    CREATED
spotesify-api    spotesify-api    latest     linux/amd64    108bd0364b23    50.5MB    8 days ago
spotesify-db    redis          7-alpine   linux/amd64    8b81dd37ff02    17.3MB    2 months ago
spotesify-nginx    nginx          alpine     linux/amd64    582c496ccf79    26MB      4 weeks ago
spotesify-web    spotesify-web    latest     linux/amd64    35d80a57e8a7    26MB      2 weeks ago
spotesify-worker    spotesify-worker    latest     linux/amd64    fb922437ac4c    49MB      11 days ago
spotesify
```

Concepto de SS00:

Aislamiento de procesos, privilegios, usuarios, permisos y optimización de imágenes de contenedores

E7 — Healthcheck operativo

Sintoma:

Al principio el healthcheck de la api no era operativo ya que la api no se mantenía estable y no podía verificar correctamente la conexión con redis. Por lo tanto el servicio no podía considerarse sano

Diagnostico:

El /api/health de la api realiza una comprobación mediante `redis.ping()`. Por tanto si la api no estaba viva, no podía resolver el servicio db o si fallaba la autenticación con redis, el healthcheck devolvía error

Solución:

Al corregir la configuración de la api, aumentando su memoria y configurando correctamente `REDIS_PASS`, el /api/health empezó a devolver el estado ok

```
alumno@spotesify-vm ~/$ spotesify on main
> curl http://localhost/api/health
{"status":"ok","redis":"connected","uptime":8969.739651737,"timestamp":"2026-05-09T12:34:40.067Z"}
```

Concepto de SS00:

Supervisión de procesos, comprobación de disponibilidad y gestión del ciclo de vida de servicios

E8 — Worker inestable

Sintoma:

El checker indicaba que el worker tenía varios reinicios y que era inestable

Diagnostico:

La causa era un límite de memoria demasiado bajo en el contenedor worker. Se había configurado con un límite de 10m, la cual era una cantidad insuficiente y al superar ese límite el proceso era finalizado

Solución:

Se aumentó el límite de memoria de 10MB a 128MB en `docker-compose.yml`. Después se volvió a contruir y lanzar los contenedores

```
alumno@spotesify-vm ~/$ spotesify on main
> docker inspect spotesify-worker --format 'RestartCount={{.RestartCount}} ExitCode={{.State.ExitCode}} OOMKilled={{.State.OOMKilled}} Memory={{.HostConfig.Memory}}'
RestartCount=0 ExitCode=0 OOMKilled=false Memory=134217728
```

Concepto de SS00:

Gestión de memoria, límite de recursos en contenedores y supervisión de procesos

E9 — Arranque no resiliente

Sintoma:

El checker indicaba arranque no resiliente. Aunque los servicios arrancaban correctamente, era posible de que api y worker no esperaran a que redis estuviera preparado antes de intentar conectarse

Diagnostico:

En docker compose, `depends_on` solo garantiza el orden de creación de contenedores, pero no que el servicio dependiente esté listo para aceptar conexiones. Por tanto api y worker podían arrancar antes de que db estuviera operativo

Solución:

Se añadió un healthcheck en db, después se modificaron los servicios api y worker para depender de db con condición: service_healthy y finalmente se recrearon los contenedores

```
alumno@spotesify-vm ~/$ spotesify on main
> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED    STATUS    PORTS
spotesify-api       spotesify-api       "docker-entrypoint.s..." api        2 days ago    Up 3 hours    (healthy)    5000/tcp
spotesify-db        redis:7-alpine      "docker-entrypoint.s..." db         2 days ago    Up 3 hours    (healthy)    6379/tcp
spotesify-nginx     nginx:alpine        "/docker-entrypoint.s..." nginx      2 days ago    Up 3 hours    0.0.0.0:80->80/tcp, [::]:80->80/tcp
spotesify-web       spotesify-web       "/docker-entrypoint.s..." web        2 days ago    Up 3 hours    80/tcp, 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
spotesify-worker    spotesify-worker    "docker-entrypoint.s..." worker     2 days ago    Up 3 hours
```

Concepto de SSOO:

Orquestación de servicios, dependencias entre procesos, disponibilidad de servicios, healthcheck y orden de arranque

FASE 2 - MEJORA DE LA INFRAESTRUCTURA

2.1 Optimización del Dockerfile

El Dockerfile inicial de la API utilizaba una imagen pesada (node:20) y ejecutaba la aplicación como usuario root.

Esto no es recomendable ya que aumenta el tamaño de la imagen y supone un riesgo de seguridad. Por ello se sustituyó la imagen base por node:20-alpine.

Además se reorganizó el Dockerfile, primero se copian los archivos package.json y package-lock.json, después se instalan las dependencias y finalmente se copia el resto del código fuente.

También se configuró el contenedor para ejecutarse con un usuario no privilegiado. En este caso el usuario node, ya incluido en la imagen de Node.

Se añadió un HEALTHCHECK funcional que consulta el endpoint /api/health de la API. Esto comprueba que la aplicación está viva y que pueda comunicarse con Redis.

```
alumno@spotesify-vm ~/$ spotesify on main
> docker compose images
docker compose exec api whoami
CONTAINER    REPOSITORY    TAG    PLATFORM    IMAGE ID    SIZE    CREATED
spotesify-api    spotesify-api    latest    linux/amd64    108bd0364b23    50.5MB    8 days ago
spotesify-db        redis          7-alpine    linux/amd64    8b81dd37ff02    17.3MB    2 months ago
spotesify-nginx     nginx          alpine     linux/amd64    582c496ccf79    26MB     4 weeks ago
spotesify-web       spotesify-web    latest     linux/amd64    35d80a57e8a7    26MB     2 weeks ago
spotesify-worker    spotesify-worker    latest     linux/amd64    fb922437ac4c    49MB     11 days ago
spotesify
```

2.2 Versionado con Git

Se inicializó un repositorio Git en el directorio del proyecto ~/spotesify. También se creó un .gitignore para evitar archivos innecesarios como logs, archivos temporales, informes generados y otros archivos del entorno de desarrollo.

El uso de Git permitió registrar de forma progresiva las correcciones realizadas durante la práctica. Cada error se documentó mediante un commit descriptivo, indicando que se había arreglado y por qué.

```

alumno@spotesity-vm .../Practica4-SS0011 on main
> git log --oneline --graph --decorate
* ec3e2c3 (HEAD -> main, origin/main, origin/HEAD) Proyecto Completado
* 26d3bfc Informe terminado
* d3b31d9 Proyecto completo y arreglado
* 1301af9 Correccion de ansible
* a9be9e5 Dockerfile optimizado
* c6bdb74 Automatizacion con ansible
* d71228b gitignore
* ea504c8 Checker report
* 28ac2af Solucion de problemas E7 E8 E9
* 2f39e02 Solucion de problemas E4 E5 E6
* c238bba Solucion de problemas E1 E2 E3
* 8a67226 Initial commit

```

2.3 Automatizacion con Ansible

Para automatizar el despliegue de SpotESIfy se creo un plyabook de Ansible llamado `deploy_spotesity.yml`, junto con un inventario (`inventory.ini`). El objetivo del playbook es poder dejar la aplicacion funcionando sin tener que ejecutar manualmente todos los comandos de instalacion y despliegue.

El playbook instala primero los paquetes necesarios y otros componentes necearios. Despues configura el repositorio de Docker, instala Docker Engine y el plugin de Docker Compose y asegura que el Docker queda arrancado y habilitado.

Durante la ejecucion aparecio un conflicto entre paquetes de Docker y paquetes procedentes del repositorio oficial de Docker. `docker-compose-v2` y `docker-compose-plugin` intentaban instalar el mismo binario. Para resolverlo, se añadio una tarea que elimina paquetes conflictivos antes de instalar los paquetes oficiales de Docker.

A continuacion, el playbook crea el directorio `/opt/spotesity`, comprueba que el proyecto origen contiene el archivo `docker-compose.yml` y copia los archivos del proyecto.

Despues, Ansible ejecuta Docker Compose construyendo las imagenes y levantando los servicios. Por ultimo verifica que la aplicacion ha arrancado correctamente realizando peticiones HTTP a `/api/health` y al catalogo `/api/songs`

Ejecucion del playbook:

```

PLAY RECAP *****
localhost          : ok=17  changed=3  unreachable=0  failed=0  skipped=1  rescued=0  ignored=0

```

El playbook se diseño para ser idempotente. Puede ejecutarse mas de una vez sin provocar errores ni cambios innecesarios. Al ejecutar el playbook una segunda vez, las tareas que ya estaban en el estado correcto aparecieron como `ok` y solo se marcaron como `changed` aquellas que realmente necesitaban actualizar algo.

Checker:

Máquina: `spotesity-vm` MAC: `08:00:27:b5:97:ea`
Fecha: `2026-05-09 12:50:37`

Verificaciones

- ✓ [E1] Web accesible a través del proxy
Flag: `2c20d73b84718b8d`
- ✓ [E2] API conectada a la base de datos
Flag: `fb3747b85913bf93`
- ✓ [E3] Catálogo de canciones accesible (5 canciones)
Flag: `0ce0cd74fd0adcde`
- ✓ [E4] Streaming de audio funcional
Flag: `5389a312ccb63202`
- ✓ [E5] Contenedor API estable (0 reinicios)
Flag: `f1f41e586686540a`
- ✓ [E6] Imagen optimizada (48MB, user: spotesity)
Flag: `eed588f440837440`
- ✓ [E7] Healthcheck operativo
Flag: `3ba1c2ca7ab4119e`
- ✓ [E8] Worker estable (0 reinicios)
Flag: `59b6d7f6fd83fb7a`
- ✓ [E9] Arranque resiliente configurado
Flag: `be10d64701986ff2`

Progreso: [] 9/9

🎉 SpotESIfy está en producción. Fase 1 completada.

📄 Informe: `/home/alumno/checker_report.txt`